

The GenLang Book

Learning, usage, and technical reference for version 0.1.0

Alihan Karakuş
September 2026

Preface

- How to read it
- What 0.1.0 is

Chapter 1 — What GenLang is

- Why it exists
- Design principles
- When to use it

Chapter 2 — A first document

Chapter 3 — Core concepts

Chapter 4 — Lexical structure

- Comments
- Identifiers
- Keywords
- Numbers
- Strings
- Punctuation
- What is not in the language

Chapter 5 — Grammar (documents)

- Objects versus lists
- `veri` shapes
- References

Chapter 6 — Values and paths

- Value kinds
- Paths

Chapter 7 — Optional schemas

Chapter 8 — Imports (`iceaktar`)

Chapter 9 — CLI

Chapter 10 — REPL and queries

Chapter 11 — Embedding: C ABI

- Lifecycle
- Ownership
- Parse versus load
- Errors
- Threading
- CMake / pkg-config

Chapter 12 — Language bindings

Chapter 13 — Architecture (for embedders)

Chapter 14 — Interchange formats

- Canonical GenLang text

- JSON and YAML
- Compact binary (GLB v1)

Chapter 15 — Semantic rules and mistakes

- Common mistakes

Chapter 16 — Security and resource limits

Chapter 17 — Cookbook

- Catalog with tags (packages, tools)
- Media library
- Notes and documents
- Unicode names
- Split a taxonomy across files

Appendix A — File checklist

Appendix B — Examples in this repository

Appendix C — Documentation map

Appendix D — License

Preface

This book is the canonical description of GenLang 0.1.0. If another document disagrees with it, this book wins, and the other document should be corrected.

It is written for people who will **write** `.gl` files, **embed** `libgenlang`, or **teach a model** to generate valid GenLang. It is not a marketing page. Every rule here matches `include/genlang.h`, `docs/grammar.md`, and the recursive-descent parser in `src/parser/parser.c`.

How to read it

You want...	Start at
To write your first file	Chapter 2, then 3
The type / set distinction	Chapter 1
Exact syntax	Chapters 4–6
Schemas and imports	Chapters 7–8
CLI, queries, REPL	Chapters 9–10
To embed the library	Chapters 11–13
JSON / YAML / binary	Chapter 14
Errors and limits	Chapters 15–16
A checklist before you ship a file	Appendix A

Companion files in the repository:

- [README.md](#) — project overview (English)
- [docs/usage.md](#) — short bilingual tutorial
- [docs/grammar.md](#) — EBNF
- [docs/api.md](#) — C ABI
- [docs/embedding.md](#) — language bindings
- [docs/architecture.md](#) — internals
- [docs/ai-guide.md](#) — instructions for language models
- [examples/](#) — runnable `.gl` samples

Keywords in the language are Turkish ASCII (`cins`, `tur`, `kume`, `veri`, `uye`, `iceaktar`). Identifiers and string values are UTF-8. This book uses English for explanation and GenLang for examples.

What 0.1.0 is

A complete, embeddable MVP: parse, validate, query, serialize, bind. Later work (WebAssembly, a documented ABI stability policy, concurrent read-only documents) stays behind the same C ABI. Items on the roadmap are intent, not a ship date.

Chapter 1 — What GenLang is

GenLang is a **declarative data language** and an **embeddable C library**. A `.gl` file declares types, sets, named values, and memberships. The library turns that text into a `GenDocument` you can query.

It is **not**:

- a replacement for JSON or YAML as a general interchange format
- a programming language (no functions, loops, or evaluation of expressions beyond data)
- a database, an ORM, or SQL
- a template, markup, or configuration DSL that runs code

The library is the product. The `genlang` executable is a thin consumer of `include/genlang.h`. Bindings in Python, Rust, Go, Node.js, Java, and C# talk only to that header plus `libgenlang`. Never invent a second runtime.

```
libgenlang = core product    (opaque C ABI)
genlang    = CLI frontend    (include/genlang.h only)
```

Why it exists

JSON can represent `boncuk` as an object. It cannot natively say all three of the following at once, as first-class relations:

1. `Kedi` is a species of `Memeli`, which is a genus under `Hayvan`, which is under `Canli`
2. `boncuk` is an instance of type `Kedi`
3. `boncuk` also belongs to the sets `EvcilHayvanlar` and `SiyahHayvanlar`

Those are different relations. GenLang keeps them separate on purpose.

Type hierarchy (SUBTYPE_OF / TYPE_OF)

```
Kedi → Memeli → Hayvan → Canli
```

Set membership (MEMBER_OF), independent of types

```
boncuk ∈ EvcilHayvanlar
```

```
boncuk ∈ SiyahHayvanlar
```

An entity can therefore report two lists that must never be merged:

```
Types:
  Kedi
  Memeli
  Hayvan
  Canli

Sets:
  EvcilHayvanlar
  SiyahHayvanlar
```

Hard rule: type ancestry is never inferred from sets. Set membership is never inferred from types. Being a `Kedi` does not put `boncuk` in any set. Belonging to `EvcilHayvanlar` does not make that set a type.

Design principles

1. Library first, CLI second.
2. Opaque C ABI so other languages bind without rewriting the runtime.
3. Type hierarchy and set membership stay distinct.
4. No silent coercion: a reference is not a copy; a string is not a number; `1` is not `1.0`.
5. Explicit ownership; no global library state.
6. Parsing does not execute code, touch the network, or invoke a shell. `iceaktar` may read local `.gl` files only.

When to use it

Use GenLang next to an application when you need:

- a taxonomy (what something *is*) and tags/channels/playlists (what group it *belongs to*) in the same document
- nested structured values with paths such as `x.a[1].b[2]`
- validation of typed records against optional type-property schemas
- a small embeddable runtime in C, with bindings

Do not use it as a universal document store, a secret vault, or a replacement for JSON APIs.

Chapter 2 — A first document

Save this as `animals.gl` (it is also `examples/animals.gl`):

```
cins Canli
cins Hayvan -> Canli
cins Memeli -> Hayvan

tur Kedi -> Memeli
tur Kopek -> Memeli

kume EvcilHayvanlar
kume SiyahHayvanlar

veri boncuk : Kedi {
  isim = "Boncuk"
  yas = 4
  renk = "siyah"
}

veri karamel : Kopek {
  isim = "Karamel"
  yas = 7
  renk = "kahverengi"
}

uye boncuk -> EvcilHayvanlar
uye boncuk -> SiyahHayvanlar
uye karamel -> EvcilHayvanlar
```

Build and check:

```
cmake -S . -B build && cmake --build build
./build/genlang check examples/animals.gl
./build/genlang query examples/animals.gl boncuk.yas
./build/genlang examples/animals.gl
```

Expected query output is the integer `4`. The summary prints type, set, and entity counts.

Open a REPL:

```
./build/genlang repl examples/animals.gl
```

Useful lines:

```
dyaz boncuk
goster boncuk
ustler Kedi
uyeler EvcilHayvanlar
icerir SiyahHayvanlar boncuk
boncuk.renk
```

`dyaz boncuk` prints **Types** and **Sets** as two headings. That layout is part of the language contract, not a CLI quirk.

Chapter 3 — Core concepts

Concept	Keyword	English	Meaning
Genus	<code>cins</code>	genus	A general type in the hierarchy
Species	<code>tur</code>	species	A more specific type
Set	<code>kume</code>	set	An unordered collection of members
Data	<code>veri</code>	entity / value	A named value, optionally typed
Membership	<code>uye</code>	member of	Independent <code>MEMBER_OF</code>
Import	<code>iceaktar</code>	import	Splice another local <code>.gl</code> file

`cins` and `tur` live on the **same** type graph. The distinction is documentary (general versus specific). Both use `->` for `SUBTYPE_OF`. A `tur` may have children. A parent may be either kind.

Sets are **not** types. You cannot write `tur Kedi -> EvcilHayvanlar`. You cannot write `uye boncuk -> Memeli`. The left of `uye` is an entity; the right is a set.

Names:

- Types share one namespace (`cins` + `tur`). Duplicate type names are `GEN_ERR_DUPLICATE`.
- Sets share another namespace.
- Entities (`veri`) share a third.
- A type and an entity *may* share a spelling. Avoid it; queries become confusing.

Declaration order in the file is free for name resolution: all types are collected first, so a parent may appear later. Memberships are resolved after all entities and sets exist. References (`@name`) are checked after every entity has been created, so forward references to entities work.

Canonical serialization order is always: **types, then sets, then entities, then memberships**, each in declaration order. `iceaktar` lines disappear after merge.

Chapter 4 — Lexical structure

Source is UTF-8. The lexer runs before the parser. Whitespace and `#` line comments are discarded.

Comments

```
# this is a comment to end of line
cins Canlı  # trailing comment
```

There are no block comments (`/* */`).

Identifiers

ASCII form: `[A-Za-z_][A-Za-z0-9_]*`.

Non-ASCII: any UTF-8 scalar value that is not a C0/C1 control, not U+00A0–style space, and not ASCII punctuation. Letters such as `ı`, `ö`, `ş`, `Canlı` are valid. Digits cannot start an identifier. Hyphens are **not** allowed in names (`kedi-001` is invalid as an identifier; it is fine inside a string).

Keywords are reserved and cannot be identifiers.

Keywords

Document keywords:

```
cins  tur  kume  veri  uye  iceaktar
```

Literals (also reserved):

```
true  false  null
```

REPL-only (illegal in `.gl` files):

```
dyaz goster uyeler icerir ustler altlar
yol ara liste yardim temizle cikis
```

If you emit a document file, never include REPL commands.

Numbers

- Integers: optional `-`, then digits. Stored as `int64_t`. Overflow is a lex/parse error path; stay in range.
- Floats: must contain a `.` **or** an exponent (`1.5`, `1e10`, `-2.0e-3`). Stored as `double`.
- `1` is an integer. `1.0` is a float. Schema matching uses **value kind**, so they are not interchangeable.

Strings

Double quotes only. Escapes: `\`, `\\`, `\n`, `\t`, `\r`. No `\'`, no `\uXXXX`, no raw strings, no interpolation. Strings must be valid UTF-8.

```
veris = "satır\nyenisi"
```

Punctuation

```
{ } [ ] = : , -> @
```

`->` is a single token (arrow). A lone `-` starts a number.

What is not in the language

No semicolons as statement terminators. No `:` between object keys and values (that is JSON). No single-quoted strings. No hexadecimal numbers. No `true` as a string unless quoted.

Chapter 5 — Grammar (documents)

A `.gl` file is a sequence of declarations, then end of file.

```

program          = { declaration } EOF ;

declaration      = genus-decl | species-decl | set-decl
                  | data-decl | membership-decl | import-decl ;

genus-decl       = "cins" ident [ "->" ident ] [ object ] ;
species-decl     = "tur"  ident [ "->" ident ] [ object ] ;
set-decl         = "kume" ident [ object ] ;

data-decl        = "veri" ident [ ":" ident ] ( object | "=" expression ) ;
membership-decl  = "uye"  ident "->" ident ;
import-decl      = "iceaktar" string ;

expression       = object | list | literal | reference ;
literal          = string | integer | float | boolean | null ;
reference        = "@" ident ;

list             = "[" [ expression { "," expression } [ "," ] ] "]" ;
object           = "{" { ident "=" expression [ "," ] } "}" ;

```

Objects versus lists

- Object keys are **identifiers**, not quoted strings.
- Object fields may be separated by newlines and **optional** commas.
- Duplicate keys in one object are `GEN_ERR_DUPLICATE`.
- List elements **require** commas. A trailing comma is allowed.

```

# correct
veri p {
    isim = "Boncuk"
    yas = 4
}

# wrong (JSON)
veri p {
    "isim": "Boncuk"
}

```

veri shapes

```
veri numbers = [10, 20, 30, 40]
veri ratio = 1.5
veri owner = @ahmet
veri person {
  name = "Ahmet"
  active = true
  nickname = null
}
veri boncuk : Kedi {
  isim = "Boncuk"
}
```

Typed `veri` uses `: Type` then either an object or `= expression`. If the type (or an ancestor) has a schema, the value **must** be an object matching that schema. Untyped `veri` is valid.

References

```
veri ahmet : Kisi { isim = "Ahmet" }
veri owner = @ahmet
veri proje { sahibi = @ahmet }
```

`@ahmet` is `GEN_VALUE_REFERENCE`, not a string and not a copy of Ahmet's fields. The target must be an **entity**. A missing target is `GEN_ERR_SEMANTIC` (unknown reference '@...'). There are no optional or weak refs.

To read the target's fields, query the entity name (`ahmet.isim`) or resolve the ref name in the host language.

Chapter 6 — Values and paths

Value kinds

Kind	Syntax	C enum
null	<code>null</code>	<code>GEN_VALUE_NULL</code>
bool	<code>true</code> / <code>false</code>	<code>GEN_VALUE_BOOL</code>
int	<code>4</code>	<code>GEN_VALUE_INT</code>
float	<code>1.5</code>	<code>GEN_VALUE_FLOAT</code>
string	<code>"Boncuk"</code>	<code>GEN_VALUE_STRING</code>
list	<code>[1, 2]</code>	<code>GEN_VALUE_LIST</code>
object	<code>{ a = 1 }</code>	<code>GEN_VALUE_OBJECT</code>
reference	<code>@ahmet</code>	<code>GEN_VALUE_REFERENCE</code>

Accessors that are asked for the wrong kind return a zero / `NULL` rather than converting. There is no implicit stringify.

Lists are ordered and zero-based. Out-of-range access is `GEN_ERR_INDEX`.

Paths

Used by `genlang query`, the REPL, `gen_get`, and `gen_eval_path`:

```
path = ident { "." ident | "[" integer "]" } ;
```

Negative indexes are rejected.

```
ver i x {
  a = [
    { b = [10, 20, 30] },
    { b = [40, 50, 60] }
  ]
}
```

`x.a[1].b[2]` evaluates to `60`. Sample: `examples/nested.gl`.

The root name must be an **entity**. You cannot start a path at a type or a set. Indexing a non-list or selecting a field on a non-object is `GEN_ERR_TYPE`. A missing field is `GEN_ERR_NOT_FOUND`.

There is no hard-coded path depth beyond the context limit `max_nesting_depth` (default 256).

Paths are not declarations. Do not put `boncuk.yas` in a `.gl` file except inside comments.

Chapter 7 — Optional schemas

If a `cins` or `tur` has a **non-empty property object**, that object is a schema for typed `veri` of that type (and of descendants, unless a child overrides a key).

Rules:

1. The instance must be an **object** (`veri a : T = 1` fails if `T` has a schema).
2. Every schema key on the type **and ancestors** must be present.
3. Value **kind** must match the sample (`int` versus `float` versus `string`, and so on). The sample payload is a kind template, not an enum of allowed values.
4. Extra keys on the instance are allowed.
5. A child type overrides the same key; the more specific type wins.
6. Types with no property object, and no ancestor schema, impose no extra constraints.
7. Nested object schemas recurse. A list schema with at least one element uses **the first element** as the item template for every instance element.

```
cins Kayit {
  id = ""
}

cins Hayvan -> Kayit {
  isim = ""
  yas = 0
}

tur Kedi -> Hayvan {
  renk = ""
}

veri boncuk : Kedi {
  id = "kedi-001"
  isim = "Boncuk"
  yas = 4
  renk = "siyah"
  ekstra = true
}
```

This is `examples/schema.gl`. `yas = 0` in the schema requires an **integer**. `yas = 4.0` would fail.

Set property objects are **metadata on the set**, not schemas for members.

```
kume EvcilHayvanlar {
  aciklama = "evde yaşayan hayvanlar"
}
```

That does not force members to have an `aciklama` field.

Chapter 8 — Imports (iceaktar)

```
iceaktar "types.gl"
tur Kedi -> Memeli
```

Rule	Detail
Path	relative to the importing file
Suffix	must be <code>.gl</code>
URLs	rejected (<code>://</code> in the path)
Empty path	<code>GEN_ERR_SEMANTIC</code>
Once	include-once; diamond imports are not duplicated
Cycles	<code>GEN_ERR_CYCLE</code>
Execution	never; only text is parsed
In-memory parse	<code>gen_document_parse</code> / binding <code>parse()</code> reject imports (<code>GEN_ERR_IO</code>)
How to import	<code>gen_document_load_file</code> or <code>gen_document_parse_at(ctx, source, origin_path, &doc)</code>
Serialize	merged document; no <code>iceaktar</code> lines

Sample tree: `examples/import/`.

`types.gl`:

```
cins Canli
cins Hayvan -> Canli
cins Memeli -> Hayvan
```

`main.gl`:

```
iceaktar "types.gl"
tur Kedi -> Memeli
veri boncuk : Kedi {
    isim = "Boncuk"
}
```

Wrong:

```
iceaktar "https://example.com/x.gl"
iceaktar "notes.txt"
```

In the REPL, `iceaktar` still needs a file origin. Load a file first (`genlang repl file.gl`) rather than starting from an empty buffer.

Chapter 9 — CLI

`genlang` never implements parser or runtime logic. It only calls the public API.

```
genlang <file.gl>           Parse, validate, summarize
genlang check <file.gl>     Validate only
genlang query <file.gl> <path> Evaluate a nested path
genlang convert <file.gl> --json
genlang convert <file.json> --from-json
genlang convert <file.gl> --yaml
genlang convert <file.yaml> --from-yaml
genlang convert <file.gl> --binary
genlang convert <file.bin> --from-binary
genlang format <file.gl>
genlang format --in-place <file.gl>
genlang repl [file.gl]
genlang --help
genlang --version
```

Exit codes:

Code	Meaning
0	success
1	general / usage
2	lex / parse
3	semantic (<code>GEN_ERR_SEMANTIC</code> , <code>GEN_ERR_CYCLE</code> , <code>GEN_ERR_DUPLICATE</code>)
4	I/O

`--binary` writes **exact bytes** (magic `GLB` + version `1`). Do not add a trailing newline to binary output. Text conversions print a trailing newline.

`format` emits canonical GenLang (types, sets, entities, memberships). `--in-place` rewrites the file via `gen_document_save_file`.

JSON import accepts the document schema in Chapter 14. YAML import uses the same schema; JSON is valid YAML 1.2 input. Block scalars (`|` , `>`) are not supported.

Chapter 10 — REPL and queries

The REPL parser accepts a **declaration**, a **command**, or a **path**. Commands are not legal in document files.

Command	Meaning	C API
<code>dyaz name</code>	types and sets of an entity, separate headings	<code>gen_format_dyaz</code>
<code>goster name</code>	formatted value	<code>gen_format_goster</code>
<code>uyeler Set</code>	members	<code>gen_set_members</code>
<code>icerir Set entity</code>	membership test	<code>gen_is_member</code>
<code>ustler Type</code>	ancestors, excluding self	<code>gen_ancestors_of</code>
<code>altlar Type</code>	descendants	<code>gen_descendants_of</code>
<code>yol A B</code>	path in the type graph if one type is an ancestor of the other	<code>gen_type_path</code>
<code>ara text</code>	case-sensitive substring over names and nested string values; sorted	<code>gen_search</code>
<code>liste cins\ tur\ kume\ veri</code>	names in declaration order	listing getters
<code>yardim</code>	help	CLI only
<code>temizle</code>	ANSI clear screen	CLI only
<code>cikis</code>	exit	CLI only
<code>x.a[1].b[2]</code>	evaluate path	<code>gen_eval_path</code>

An empty REPL concatenates declarations and re-parses them with `gen_document_parse` (so `iceaktar` still needs an origin).

`gen_types_of(entity)` returns the declared type, then ancestors. `gen_ancestors_of(type)` does **not** include self. `gen_descendants_of` walks children recursively in declaration order.

`gen_search` uses `strstr` (case-sensitive). It matches type names, set names, entity names, and string values nested inside entities. Results are sorted.

Chapter 11 — Embedding: C ABI

Header: `include/genlang.h`. All structs are opaque. Link `libgenlang`. No exceptions: every fallible call returns `GenResult`.

Lifecycle

```
create context → load or parse → query / convert → free results
→ free document → free context
```

```
#include <genlang.h>
#include <stdio.h>

int main(void)
{
    GenContext *ctx = gen_context_create();
    GenDocument *doc = NULL;
    GenValue *value = NULL;
    GenQueryResult *types = NULL;

    if (gen_document_load_file(ctx, "examples/animals.gl", &doc) != GEN_OK) {
        fprintf(stderr, "%s\n", gen_error_message(gen_context_last_error(ctx)));
        gen_context_free(ctx);
        return 1;
    }

    if (gen_get(doc, "boncuk.yas", &value) == GEN_OK) {
        printf("yas = %lld\n", (long long)gen_value_int(value));
        gen_value_free(value);
    }

    if (gen_types_of(doc, "boncuk", &types) == GEN_OK) {
        for (size_t i = 0; i < gen_query_result_count(types); i++) {
            printf("%s\n", gen_query_result_name(types, i));
        }
        gen_query_result_free(types);
    }

    gen_document_free(doc);
    gen_context_free(ctx);
    return 0;
}
```

A slightly larger sample is `examples/embed.c`.

Ownership

OWNED (you free)	BORROWED (until owner dies)
<code>GenContext *</code>	names from type/set/entity getters
<code>GenDocument *</code>	<code>gen_entity_value</code> , nested list/object items
<code>GenValue *</code> from <code>gen_get</code> / <code>gen_eval_path</code> / <code>gen_value_clone</code>	<code>gen_value_string</code> , <code>gen_value_reference</code>
<code>GenQueryResult *</code>	<code>gen_error_message</code> , <code>gen_error_path</code>
<code>serialize</code> / <code>JSON</code> / <code>YAML</code> / <code>binary</code> / <code>format</code> buffers (<code>gen_string_free</code>)	<code>gen_version()</code>

Do not `gen_value_free` a pointer from `gen_entity_value`. Bindings **must copy** borrowed UTF-8 before `gen_document_free`.

Parse versus load

Function	iceaktar
<code>gen_document_parse</code> / <code>parse_n</code>	reject (no origin)
<code>gen_document_parse_at(..., origin_path)</code>	resolve relative to <code>origin_path</code>
<code>gen_document_load_file</code>	origin is the file path

`parse_n` takes an explicit length; the buffer need not be NUL-terminated beyond `length`.

Errors

Lexer and parser stop at the first failure. The semantic analyzer may record **several** (`gen_context_error_count` / `gen_context_error`). `gen_context_last_error` points at the **first** slot; later errors do not overwrite it.

Line and column are 1-based. `gen_error_path` is `NULL` for in-memory sources.

`GenResult` numeric values

These match the enum order in the header:

```
0  GEN_OK
1  GEN_ERR_INVALID_ARGUMENT
2  GEN_ERR_OUT_OF_MEMORY
3  GEN_ERR_IO
4  GEN_ERR_LEX
5  GEN_ERR_PARSE
6  GEN_ERR_SEMANTIC
7  GEN_ERR_NOT_FOUND
8  GEN_ERR_TYPE
9  GEN_ERR_INDEX
10 GEN_ERR_DUPLICATE
11 GEN_ERR_CYCLE
12 GEN_ERR_INVALID_OPERATION
13 GEN_ERR_SERIALIZATION
14 GEN_ERR_QUERY
```

A parse of `cins\n` yields code 5.

Threading

No global mutable library state. Distinct `GenContext` objects may be used from different threads. Do not mutate the same `GenDocument` concurrently. Concurrent **read-only** access is **not** a 0.1.0 contract.

CMake / pkg-config

```
find_package(GenLang REQUIRED)
target_link_libraries(my_app PRIVATE GenLang::genlang)
```

```
pkg-config --cflags --libs genlang
```

Requires CMake 3.16+ and a C17 compiler. `uthash` is vendored and is not part of the public API.

Chapter 12 — Language bindings

All first-class wrappers live under `bindings/` and copy values into host types.

Language	Path	Mechanism
C / C++	<code>include/genlang.h</code>	public ABI
Python	<code>bindings/python</code>	<code>cffi</code> ABI mode
Rust	<code>bindings/rust</code>	Drop wrappers
Go	<code>bindings/go</code>	<code>cgo</code>
Node.js	<code>bindings/node</code>	N-API add-on <code>genlang.node</code>
Java	<code>bindings/java</code>	JNA, <code>AutoCloseable</code>
C#	<code>bindings/csharp</code>	<code>P/Invoke</code> , <code>IDisposable</code>

Need `iceaktar`? Call **load**, not `parse`. Set `LD_LIBRARY_PATH` (or `GENLANG_LIB_DIR` / `jna.library.path`) to the directory that contains `libgenlang.so`.

Python:

```
import genlang
with genlang.load("examples/music.gl") as doc:
    print(doc.get("parca_01.baslik"))
    print(doc.members("Gece"))
```

Runnable sample: `examples/embed_python.py`.

```
PYTHONPATH=bindings/python/src GENLANG_LIB_DIR=build python3 examples/embed_python.py
```

Java:

```
try (genlang.Document doc = genlang.Document.parse(src)) {
    doc.get("boncuk.yas"); // Long
}
```

Copied host values stay valid after `close()`. Do not keep raw C pointers.

The language server `lsp/genlang_lsp.py` speaks LSP over stdio (diagnostics, hover, definition) and uses the Python bindings.

```
PYTHONPATH=bindings/python/src GENLANG_LIB_DIR=build python3 lsp/genlang_lsp.py
```

Chapter 13 — Architecture (for embedders)

```

Source → Lexer → Parser → AST → iceaktar expansion
                                → Semantic analyzer → GenDocument
                                    /      |      \
                                Type graph Sets Values
                                    \      |      /
                                    Query → Serialization

```

The AST is discarded after the document is built. Runtime objects do not alias parser nodes. JSON, YAML, and binary converters walk the runtime document. Import of those formats emits GenLang text internally and parses it, so semantic rules still apply.

Internal relation tags: `SUBTYPE_OF`, `TYPE_OF`, `MEMBER_OF`, `HAS_PROPERTY`, `CONTAINS`, `REFERENCES`. The public ABI does not expose the graph.

The CLI must not grow parser or runtime logic. Bindings must not reimplement the language.

Chapter 14 — Interchange formats

Canonical GenLang text

`gen_document_serialize` is deterministic. Repeating it on an unchanged document yields the same bytes. Order: types, sets, entities, memberships.

Typed objects serialize as `veri name : Type { ... }`. Untyped objects as `veri name { ... }`. Other values as `veri name = ...`.

JSON and YAML

A document is always an object with **four arrays**:

```
{
  "types": [
    {
      "kind": "cins",
      "name": "Hayvan",
      "parent": "Canli",
      "properties": null
    }
  ],
  "sets": [
    { "name": "EvciHayvanlar", "properties": null }
  ],
  "entities": [
    { "name": "boncuk", "type": "Kedi", "value": { "yas": 4 } }
  ],
  "memberships": [
    { "entity": "boncuk", "set": "EvciHayvanlar" }
  ]
}
```

- `kind` is `"cins"` or `"tur"`.
- `parent` / `entity type` may be JSON `null`.
- Value references become `{"$ref": "ahmet"}` (YAML: a mapping with key `$ref`). That is **not** GenLang source.
- YAML uses the same schema. `from_yaml` also accepts JSON.

Compact binary (GLB v1)

Little-endian. Do not append a newline.

1. Magic `G L B` (`0x47 0x4C 0x42`)
2. Version byte `1`
3. Four `u32` counts: types, sets, entities, memberships
4. Each type: `u8 kind` (`0` genus, `1` species), name string, parent string (nullable), value (properties)

5. Each set: name string, value (properties)
6. Each entity: name string, type name string (nullable), value
7. Each membership: entity name string, set name string

Strings: `u32` length then bytes. Length `0xFFFFFFFF` means a null string (no parent / no type). Embedded NUL in binary payloads is allowed; `from_binary` takes an explicit length.

Values: a `u8` kind tag, then payload (bool `u8`, int `u64` bit pattern of `int64`, float IEEE-754 little-endian `f64`, string length+bytes, list count+items, object count+key/value pairs, reference as a string).

Do not hand-author binary. Round-trip through the library.

Chapter 15 — Semantic rules and mistakes

Situation	Result
Duplicate type / set / entity	GEN_ERR_DUPLICATE
Duplicate membership of the same entity in the same set	GEN_ERR_DUPLICATE
Unknown parent type, entity type, set, or @ref target	GEN_ERR_SEMANTIC
uye of a missing entity	GEN_ERR_SEMANTIC
Type cycle, including cins A -> A	GEN_ERR_CYCLE
Schema mismatch or missing required key	GEN_ERR_SEMANTIC
iceaktar without file origin	GEN_ERR_IO
iceaktar "x.txt" or URL	GEN_ERR_SEMANTIC
Missing import file	GEN_ERR_IO
Bad tokens (\$, invalid UTF-8)	GEN_ERR_LEX
Incomplete syntax (cins alone)	GEN_ERR_PARSE
Path index out of range	GEN_ERR_INDEX
Unknown path root	GEN_ERR_NOT_FOUND

Forward type parents are allowed. uye may appear before veri / kume in the file.

Common mistakes

Mistake	Fix
<code>tur Kedi -> EvcilHayvanlar</code>	Sets are not types. Use <code>uye</code> .
JSON object syntax in <code>.gl</code>	<code>key = value</code> inside <code>{ }</code> .
Quoting type names	<code>Canli</code> , not <code>"Canli"</code> .
<code>"@x"</code> when you meant a ref	Use <code>@x</code> .
<code>iceaktar</code> inside <code>parse()</code>	<code>load("file.gl")</code> or <code>parse_at</code> .
Assuming Unicode identifiers are forbidden	They are allowed; keywords stay ASCII.
<code>a[-1]</code>	Rejected.
Merging <code>dyaz</code> Types/Sets	Print two sections.
Inventing <code>extends</code> , <code>class</code> , <code>import</code>	Only the keywords in Chapter 4.
Schema: extra keys forbidden	Extra keys are allowed.
Schema: <code>1</code> matching float sample <code>1.0</code>	Kinds differ.
REPL commands in a saved <code>.gl</code> file	Documents are declarations only.
Binary convert + extra newline	Write exact length.
Keeping C string pointers after <code>close()</code>	Copy first.

Chapter 16 — Security and resource limits

Parsing does not run a shell, use the network, load dynamic code, or execute scripts. `iceaktar` may read local `.gl` files only. Untrusted files can still exhaust memory or time; use context limits.

Default `GenLimits`:

Field	Default
<code>max_source_size</code>	16 MiB
<code>max_nesting_depth</code>	256
<code>max_string_length</code>	1 MiB
<code>max_object_properties</code>	65536
<code>max_list_length</code>	65536

All fields must be greater than zero. Change them with `gen_context_set_limits` before parse. Deep nesting fails with a parse error instead of overflowing the stack.

Do not store secrets in `.gl` files that you serialize to JSON. GenLang is not an encryption format. (`prot`-style apps should keep ciphertext out of the document and use GenLang only for metadata.)

Chapter 17 — Cookbook

Catalog with tags (packages, tools)

Types = what the thing *is*. Sets = release channel and implementation tags. See `examples/packages.gl`.

```
cins Yazilim { aciklama = "" }
tur TerminalAraci -> Yazilim { dil = "" }
kume Uretim
kume Cile

veri pnot : TerminalAraci {
  aciklama = "şifreli not uygulaması"
  dil = "C"
}
uye pnot -> Uretim
uye pnot -> Cile
```

Media library

Types = album / track. Sets = playlists. See `examples/music.gl`. A track is not on a playlist until `uye` says so.

Notes and documents

Types = kind of document. Sets = `Sifreli`, `Taslak`, `Arsiv`. See `examples/notes.gl`. Forward `@refs` between notes are allowed.

Unicode names

```
cins Canlı
tur Kedi -> Canlı
veri böncü : Kedi { isim = "Böncü" }
```

See `examples/unicode.gl`. Keywords remain ASCII; you cannot rename `cins` to `genus` in source.

Split a taxonomy across files

Put genera in `types.gl`, import them, declare species and data in `main.gl`. Always **load** the importer file.

Appendix A — File checklist

Before you output a `.gl` document:

☐

Only document keywords: `cins` `tur` `kume` `veri` `uye` `iceaktar`

☐

Types and sets are different; membership is `uye`

☐

Objects use `ident = value`; lists use commas

☐

Strings are `"..."`; refs are `@ident`

☐

Typed `veri` matches ancestor schemas if present

☐

Every `@name`, `: Type`, and `uye ... -> Set` names a declared entity, type, or set

☐

Imports are `iceaktar "file.gl"` and you **load** the file

☐

No JSON, no code, no network, no negative indexes, no REPL commands

When explaining a file, describe type ancestry and set membership in **separate lists**.

Appendix B — Examples in this repository

File	Shows
<code>examples/animals.gl</code>	types, sets, typed entities, membership
<code>examples/basic.gl</code>	primitives, lists, objects
<code>examples/nested.gl</code>	<code>x.a[1].b[2]</code>
<code>examples/relations.gl</code>	@ references
<code>examples/schema.gl</code>	optional type schemas
<code>examples/unicode.gl</code>	UTF-8 identifiers
<code>examples/music.gl</code>	media kinds vs playlists
<code>examples/packages.gl</code>	tool catalog vs channels
<code>examples/notes.gl</code>	document kinds vs tags
<code>examples/import/</code>	<code>iceaktar</code>
<code>examples/embed.c</code>	C ABI queries
<code>examples/embed_python.py</code>	Python <code>load / get / members</code>
<code>examples/project.gl</code>	small module catalog

Appendix C — Documentation map

This book is the learning and usage authority. Other markdown files are views of the same facts:

File	Role
<code>README.md</code>	English project landing page
<code>docs/usage.md</code>	Short bilingual tutorial
<code>docs/grammar.md</code>	Normative EBNF (must match the parser)
<code>docs/api.md</code>	Function-by-function C ABI
<code>docs/embedding.md</code>	Bindings
<code>docs/architecture.md</code>	Pipeline and ownership internals
<code>docs/ai-guide.md</code>	Condensed rules for language models
<code>CHANGELOG.md</code>	What shipped
<code>CONTRIBUTING.md</code>	How to change the code without breaking this book

If you change the language or ABI, update this book in the same change.

Appendix D — License

GenLang is licensed under the MIT License. Copyright © 2026 Alihan Karakuş. `uthash` is vendored under its own BSD-style license in `third_party/uthash/`.

End of The GenLang Book — version 0.1.0